

Zadanie:

Cel zadania polega na implementacji algorytmu gradientu prostego oraz zbadaniu jego zachowania dla różnych wartości wymienionych niżej hiperparametrów. Metodę należy zastosować dla następujących funkcji:

- $f(x) = x_1^2 + x_2^2$
- funkcja [Matyas](#) (dla 2 wymiarów)

gdzie x_i należy do przedziału $[-10, 10]$, dla każdego $i = 1, 2$.

Kroki do wykonania:

1. Zaimplementuj algorytm gradientu prostego.
2. Zbadaj wpływ wartości parametru kroku na zbieżność metody - należy sporządzić wykres par (wartość funkcji celu, nr iteracji).
3. Dla ustalonej wartości parametru kroku zbadaj zachowanie algorytmu dla trzech wybranych punktów startowych. Wyniki przedstaw w postaci wizualnej.

Uwagi:

- Zaimplementowana metoda powinna być uniwersalna, tzn. działać dla dowolnej zadanej funkcji celu.
- Warunki stopu: maksymalna liczba iteracji, zbieżność gradientu.
- Gradient można liczyć z definicji, bądź też użyć np. modułu [autograd](#).

Wskazówka:

Do wizualizacji funkcji można użyć następującego kodu:

```
import matplotlib.pyplot as plt
import numpy as np

def visualize_fun(obj_fun: callable, trajectory: np.ndarray):
    min_x, min_y = trajectory[-1]
    MIN_X = 10
    MAX_X = 10
    PLOT_STEP = 100

    x1 = np.linspace(-MIN_X, MAX_X, PLOT_STEP)
    x2 = np.linspace(-MIN_X, MAX_X, PLOT_STEP)
    X1, X2 = np.meshgrid(x1, x2)
    Z = obj_fun(X1, X2)

    plt.figure(figsize=(8, 6))
    plt.pcolormesh(X1, X2, Z, cmap='viridis', shading='auto')
    plt.colorbar(label='Objective Function Value')
    plt.xlabel('x1')
    plt.ylabel('x2')
    plt.title('Objective Function Visualization')

    plt.scatter(min_x, min_y, color='yellow', label='Minimum found by gradient descent alg.')
    plt.plot(trajectory[:, 0], trajectory[:, 1], marker='o', color='red', label='Gradient Descent Steps', alpha=0.5)

    plt.legend()
    plt.show()
```